

CO-2
ASSESSMENT TOOL 2
Design Task

NAME: Y.SAI PRASANTH VARMA

REG NO: 192411191

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4308

QUESTIONS:

Question 1: Responsive Navigation Bar Design

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- (a) Identify key components required in a navigation bar for an e-commerce website.
- (b) Design a responsive navigation bar using appropriate CSS techniques.
- (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
- (d) Evaluate the usability and suggest improvements for better user experience.

Answer:

Scenario: An e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

(a) Key components required in a navigation bar for an e-commerce website

- Logo / brand name — links back to the homepage
- Primary navigation links — Home, Categories, Deals, New Arrivals
- Search bar — prominent, since product discovery drives e-commerce conversion
- Account/login icon — access to profile, orders, and wishlist
- Cart icon with item-count badge — persistent visibility of cart status
- Hamburger menu toggle — appears only on tablet/mobile breakpoints
- Category mega-menu / dropdown — for deep catalog navigation on desktop

(b) Responsive navigation bar using appropriate CSS techniques

The layout uses Flexbox for the bar itself (space-between alignment of logo, links, and icons) combined with media queries to progressively adapt the layout at each breakpoint.

```
/* Base (mobile-first) styles */
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.75rem 1.25rem;
```

```
background: #1f2937;
position: sticky;
top: 0;
z-index: 100;
}

.nav-links {
  display: none;      /* hidden on mobile by default */
  gap: 1.5rem;
}

.nav-icons { display: flex; gap: 1rem; align-items: center; }

.hamburger { display: block; cursor: pointer; }

/* Tablet and up */
@media (min-width: 768px) {
  .nav-links { display: flex; }
  .hamburger { display: none; }
  .search-bar { display: block; width: 240px; }
}

/* Desktop */
@media (min-width: 1200px) {
  .navbar { padding: 1rem 3rem; }
  .search-bar { width: 360px; }
  .nav-links { gap: 2.5rem; }
}
```

(c) Adapting to smaller screens (hamburger menu)

On mobile, the primary links collapse into a hidden panel toggled by a hamburger icon. JavaScript adds/removes an active class that CSS uses to slide or fade the menu into view:

```
.nav-links.active {
  display: flex;
  flex-direction: column;
  position: absolute;
  top: 100%; left: 0; right: 0;
  background: #1f2937;
  padding: 1rem;
}

const hamburger = document.querySelector('.hamburger');
const navLinks = document.querySelector('.nav-links');
hamburger.addEventListener('click', () => {
  navLinks.classList.toggle('active');
});
```

(d) Usability evaluation and improvements

- Strength: sticky positioning keeps navigation and cart access always visible while scrolling long product listings.
- Strength: mobile-first base styles keep initial payload small, improving load time on mobile networks.
- Improvement: add a visible focus outline and ARIA attributes (aria-expanded on the hamburger button) for keyboard and screen-reader users.
- Improvement: close the mobile menu automatically when a link is tapped or when the user taps outside it.
- Improvement: use CSS transitions (max-height or transform) instead of an instant display toggle, so the menu opens smoothly rather than snapping.

Question 2: Product Hover Effect

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.
- (b) Suggest appropriate CSS animation or transition for product interaction.
- (c) Design a hover effect for a product card.
- (d) Evaluate performance and user experience considerations.

Answer:

Scenario: An e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Suitable CSS properties for hover effects

- transform (scale, translateY) — lifts or zooms the card without affecting document flow
- box-shadow — adds depth, signalling that the card is interactive
- opacity — used to fade a secondary overlay (e.g., 'Add to Cart' button) into view
- filter (brightness/blur) — subtly dims the product image behind the overlay
- cursor: pointer — reinforces clickability

(b) Appropriate CSS animation / transition for product interaction

- transition is preferred over animation for hover states: it is lightweight, runs only on state change (hover in/out), and needs no keyframes.
- Apply transition: transform 0.3s ease, box-shadow 0.3s ease; on the base .product-card so both the scale and shadow animate smoothly in both directions.
- Use will-change: transform; sparingly to hint the browser to optimise the animated layer, improving smoothness on lower-end devices.

(c) Hover effect design for a product card

```
.product-card {
  position: relative;
  overflow: hidden;
  border-radius: 12px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.08);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.product-card:hover {
  transform: translateY(-6px) scale(1.02);
  box-shadow: 0 12px 24px rgba(0,0,0,0.18);
}

.product-card .quick-add {
  position: absolute;
  bottom: -48px;           /* hidden below the card */
  left: 0; right: 0;
  text-align: center;
  background: rgba(31,41,55,0.9);
  color: #fff;
  padding: 0.6rem;
```

```

transition: bottom 0.3s ease;
}

.product-card:hover .quick-add {
  bottom: 0;          /* slides up into view */
}

```

(d) Performance and UX evaluation

- Performance: animating transform and opacity only (not top/left/width) keeps the effect on the GPU compositor, avoiding layout reflow and jank.
- Performance: limit transition duration to 200–350ms; longer feels sluggish on a page with dozens of product cards.
- UX: hover-only effects are invisible on touchscreens, so the 'quick add' action should also be reachable via a always-visible icon or a tap-triggered state on mobile.
- UX: pair the animation with a subtle cursor and label change so the interactive affordance is unambiguous.

Question 3: Layout Selection (Flexbox vs Grid)

Suppose that a dashboard layout needs to be designed for a web application.

- Identify layout requirements of a dashboard (rows, columns, alignment).
- Compare Flexbox and Grid based on layout needs.
- Choose the most suitable layout technique and justify your choice.
- Evaluate the scalability and responsiveness of the chosen approach.

Answer:

Scenario: A dashboard layout needs to be designed for a web application.

(a) Layout requirements of a dashboard

- A fixed sidebar (navigation/filters) and a main content region
- A top header bar spanning the full width (branding, search, user profile)
- A grid of widget/KPI cards that must align in consistent rows and columns
- Areas of differing sizes — e.g., a large chart spanning two columns next to smaller stat cards
- Alignment and consistent gutters between all widgets regardless of content length

(b) Flexbox vs Grid comparison

Aspect	Flexbox	Grid
Dimension	One-dimensional (row OR column)	Two-dimensional (rows AND columns together)

Aspect	Flexbox	Grid
Best for	Linear arrangements: nav bars, toolbars, button groups	Overall page/dashboard skeleton with widgets that must align across both axes
Sizing control	Content-driven; sizes items based on flex-grow/shrink	Explicit tracks (fr, px, %) let widgets span multiple rows/columns precisely
Alignment across rows	Awkward — items in different rows don't auto-align into columns	Native — grid-template-columns keeps every row's columns aligned
Typical use here	Inside a single widget (icon + label) or the top header bar	The dashboard's outer skeleton (sidebar / header / widget area)

(c) Chosen layout technique and justification

CSS Grid for the outer dashboard structure, with Flexbox nested inside individual components. Grid is chosen because the dashboard is inherently two-dimensional — the sidebar, header, and widget grid must all align on both axes simultaneously, which is exactly what grid-template-areas and grid-template-columns are built for. Flexbox is then used within each widget (e.g., aligning an icon next to a stat number) where only one direction of alignment is needed.

```
.dashboard {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: 64px 1fr;
  grid-template-areas:
    'sidebar header'
    'sidebar main';
  min-height: 100vh;
}

.sidebar { grid-area: sidebar; }
.header { grid-area: header; }
.main { grid-area: main;
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 1.25rem;
  padding: 1.5rem;
}
```

```
.widget-chart { grid-column: span 2; } /* large chart spans 2 cols */

.stat-card { /* flexbox inside a widget */
  display: flex;
  align-items: center;
  gap: 0.75rem;
}
```

(d) Scalability and responsiveness evaluation

- Scalable: new widgets are added by naming a grid-area or letting auto-placement flow them into the next cell, without restructuring existing CSS.
- Responsive: grid-template-columns can switch to repeat(auto-fit, minmax(220px, 1fr)) so widgets reflow into fewer columns automatically as the viewport shrinks.
- At smaller breakpoints the grid-template-areas can be redefined (e.g., stacking sidebar above main) with a single media query, keeping markup unchanged.
- Trade-off: Grid has a steeper learning curve and is less intuitive than Flexbox for simple, single-axis components — hence the hybrid approach.

Question 4: Mobile-First Blog Layout

Suppose that a blog page must be designed following a mobile-first approach.

- (a) Identify key sections of a blog layout.
- (b) Design a mobile-first layout using CSS.
- (c) Demonstrate how the layout scales for larger screens using media queries.
- (d) Evaluate accessibility and readability of the design.

Answer:

Scenario: A blog page must be designed following a mobile-first approach.

(a) Key sections of a blog layout

- Header — blog title/logo and a collapsible navigation menu
- Featured/hero post — highlights the latest or pinned article
- Article list — title, thumbnail, excerpt, author, and date for each post
- Single-article view — title, hero image, body content, tags
- Sidebar — categories, search, popular posts (deprioritised on mobile)
- Comments section
- Footer — social links, newsletter signup, copyright

(b) Mobile-first layout using CSS

Base (unprefixed) styles target the smallest screen first — a single-column stack — and larger layouts are added only inside min-width media queries, so mobile devices never download or apply unnecessary override rules.

```

/* Mobile base: single column, sidebar stacks below content */
.blog-layout {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
  padding: 1rem;
}

.post-card { display: flex; flex-direction: column; }
.post-card img { width: 100%; height: auto; border-radius: 8px; }

.sidebar { order: 2; } /* pushed below articles on mobile */
.article-list { order: 1; }

```

(c) Scaling for larger screens using media queries

```

/* Tablet: two-column post grid */
@media (min-width: 768px) {
  .article-list {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 1.5rem;
  }
}

/* Desktop: content + sidebar side by side */
@media (min-width: 1024px) {
  .blog-layout {
    flex-direction: row;
    align-items: flex-start;
  }
  .article-list {
    grid-template-columns: repeat(3, 1fr);
    flex: 3;
  }
  .sidebar { flex: 1; order: 2; position: sticky; top: 1rem; }
}

```

(d) Accessibility and readability evaluation

- Line length: constrain article body text to max-width: 65ch so lines stay comfortably readable at any screen size.
- Contrast: ensure text/background contrast meets WCAG AA (4.5:1) — important since blogs are text-heavy.
- Font scaling: use rem units for typography so it respects the user's browser font-size preference.
- Semantic HTML: <article>, <header>, <nav>, <time> elements improve screen-reader navigation and SEO.
- Images: include descriptive alt text and use loading="lazy" for off-screen thumbnails to improve mobile performance.

Question 5: JavaScript Event Handling for Dynamic Men

Suppose that a website includes a dynamic menu that responds to user interactions.

- (a) Identify appropriate JavaScript events for menu interaction.
- (b) Design event handling logic for menu toggle functionality.
- (c) Implement interaction for user actions (click/hover).
- (d) Evaluate performance and suggest improvements.

Answer:

Scenario: A website includes a dynamic menu that responds to user interactions.

(a) Appropriate JavaScript events for menu interaction

- click — toggles the mobile menu open/closed on tap of the hamburger icon
- mouseenter / mouseleave — opens/closes desktop dropdown submenus on hover (preferred over mouseover/mouseout, which bubble and re-fire on child elements)
- keydown — supports Escape to close the menu and Tab/Arrow keys for keyboard navigation
- focus / blur — ensures the menu opens when a link inside it receives keyboard focus
- click (on document) — detects a click outside the menu to auto-close it

(b) Event handling logic design for menu toggle functionality

- A single source of truth: an isOpen boolean (or a CSS class on the menu) that all handlers read and update, avoiding conflicting states.
- Event delegation: attach one listener to a parent container instead of one per menu item, so dynamically added items still work and memory use stays low.
- Guard against event bubbling: the outside-click listener must ignore clicks that originate inside the menu, otherwise the menu closes itself immediately after opening.
- Debounce resize handling if the menu behavior needs to reset when the viewport crosses the mobile/desktop breakpoint.

(c) Implementation for user actions (click / hover)

```
const toggleBtn = document.querySelector('.hamburger');
const menu      = document.querySelector('.nav-links');
```

```

// Click: open/close on mobile
toggleBtn.addEventListener('click', (e) => {
  e.stopPropagation();
  const isOpen = menu.classList.toggle('active');
  toggleBtn.setAttribute('aria-expanded', isOpen);
});

// Click outside: close the menu
document.addEventListener('click', (e) => {
  if (menu.classList.contains('active') && !menu.contains(e.target)) {
    menu.classList.remove('active');
    toggleBtn.setAttribute('aria-expanded', 'false');
  }
});

// Escape key: close the menu
document.addEventListener('keydown', (e) => {
  if (e.key === 'Escape') menu.classList.remove('active');
});

// Desktop dropdown: hover
document.querySelectorAll('.has-dropdown').forEach((item) => {
  item.addEventListener('mouseenter', () => item.classList.add('open'));
  item.addEventListener('mouseleave', () => item.classList.remove('open'));
});

```

(d) Performance evaluation and improvements

- Performance: event delegation (single listener on a parent) scales better than per-item listeners for menus with many links.
- Performance: toggling a class and letting CSS transitions animate is cheaper than animating styles directly from JavaScript on every frame.
- Improvement: add `aria-expanded`, `aria-haspopup`, and `role="menu"` attributes so assistive technology announces the menu's open/closed state.
- Improvement: on touch devices, hover events don't fire reliably — detect touch and fall back to click/tap behavior for dropdowns.
- Improvement: throttle or debounce the resize listener (if used) so it doesn't run excessively while the window is being dragged.